Linear Regression

An ice cream shop recorded its sales on various days of the year, together with the weather on that day, to try to determine if there is a meaningful relationship between temperature and sales. The observations are plotted below, in blue. The red line is an example of a simple linear regression obtained by minimizing the total squared error (each one is shown as a red box) from the points to the line.

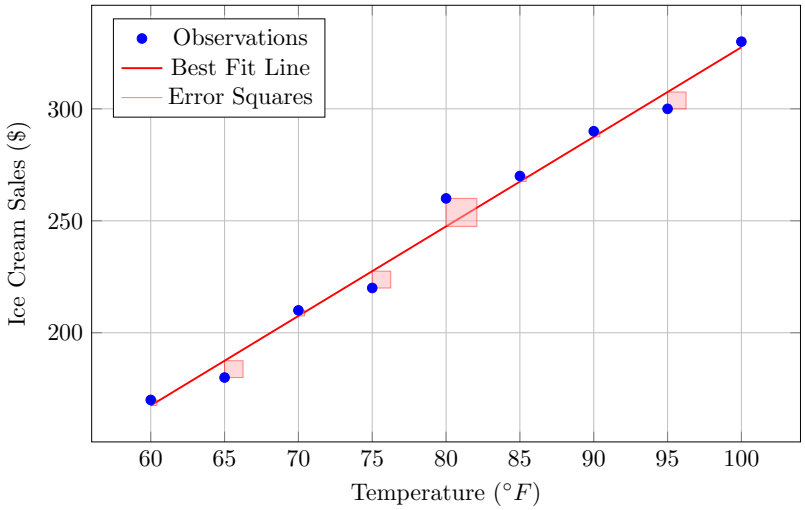


Figure 1: Temperature vs. Ice Cream Sales with the Best Fit Line

1 Ordinary Least-Squares

Imagine we are given a data set $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid \boldsymbol{x}_i \in \mathbb{R}^d, y_i \in \mathbb{R}, i \in \{1, \ldots, m\}\}$ of size m, where each observation $\boldsymbol{x}_i \in \mathbb{R}^d$ is d-dimensional, meaning each observation is described by d **features**.

In a regression model, when we "regress Y on X," Y is called the **response**, or **dependent**, variable, while the features are called the **explanatory** variables, or the **predictors**.

The feature values, which are presented as tabular data, can be encoded in a matrix $X \in \mathbb{R}^{m \times d}$, as follows:

$$X = \begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_m \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1d} \\ x_{21} & x_{22} & \ldots & x_{2d} \\ \vdots & & & \\ x_{m1} & x_{m2} & \ldots & x_{md} \end{bmatrix}$$

Each $\boldsymbol{x}_i$ in X is a row vector, i.e., an element of $\mathbb{R}^d$. When the number of features $d = 1$, the problem is called **simple regression**. Otherwise, it is called **multiple regression**.

[1] This notes were compiled in conjunction with Professor Eric Ewing.

The response variables $\boldsymbol{y} \in \mathbb{R}^m$ are given by a column vector:

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

The goal is to find a set of **weights** $\boldsymbol{w} \in \mathbb{R}^d$, or **coefficients**, s.t. the model's **estimates**,[2] or **predictions**,[3] $X\boldsymbol{w}$, approximate $\boldsymbol{y}$:

$$X\boldsymbol{w} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \dots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{md} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} = \begin{bmatrix} w_1 x_{11} + w_2 x_{12} + \ldots + w_d x_{1d} \\ w_2 x_{21} + w_2 x_{12} + \ldots + w_d x_{1d} \\ \vdots \\ w_1 x_{m1} + w_2 x_{m2} + \ldots + w_d x_{md} \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \boldsymbol{y}$$

Observe that $X\boldsymbol{w} = \boldsymbol{y}$ is a system of m equations with d unknowns. A system of equations can be solved when it has the same number of equations and unknowns, i.e., when $n = d$. If $n < d$, i.e., if the matrix is wider than it is tall, then the system is **underdetermined**; it has infinitely many solutions. If $n > d$, i.e., if the matrix is taller than it is wide, which is usually the case in regression, the system is **overdetermined**, which means it has no solution. Consequently, rather than aim to solve this system of equations exactly, we solve for weights that minimize the difference between the predicted values $X\boldsymbol{w}$ and the observed values y.

Define the **residual** of a candidate solution $\boldsymbol{w}$ as the difference between $\boldsymbol{y}$ and $X\boldsymbol{w}$, i.e.,

$$\text{residual}(\boldsymbol{w}) = \boldsymbol{y} - X\boldsymbol{w}$$

The **least squares objective** is to minimize the *square* of this residual value: $\text{loss}(\boldsymbol{w}) \doteq \text{residual}^2(\boldsymbol{w})$.

Why square the residuals? "My reasons are better than they seem."[4]

1. Squaring errors precludes positive and negative errors from canceling one another out (and so does using absolute error, based on absolute values)
2. Squaring errors penalizes larger errors far more than smaller ones, thereby forcing the model to prioritize minimizing large errors over smaller ones—which may or may not be desirable![5]
3. The ensuing estimator matches that which is derived by the maximum likelihood principle, assuming the data are normally distributed
4. This form of regression affords a closed-form solution

As $\text{residual}(\boldsymbol{w})$ is an m-dimensional column vector, we square it as follows:

$$\text{loss}(\boldsymbol{w}) = \text{residual}^2(\boldsymbol{w}) = (\boldsymbol{y} - X\boldsymbol{w})^T(\boldsymbol{y} - X\boldsymbol{w}) = \sum_{i=1}^{m} (\boldsymbol{y} - X\boldsymbol{w})_i^2$$

In other words, we are interested in minimizing the sum of the squared residuals.

[2] statistical nomenclature
[3] machine learning nomenclature
[4] A modern English translation of William Shakespeare, *All's Well that Ends Well*
[5] Minimizing absolute error is more robust to outliers, because it does not pay undue attention to outliers.

As usual, to minimize an objective, we take its derivative and set it equal to zero.

First, let's simplify the loss:

$$\begin{aligned}
&\text{loss}(\boldsymbol{w}) && (1)\\
&= (\boldsymbol{y} - X\boldsymbol{w})^T(\boldsymbol{y} - X\boldsymbol{w}) && (2)\\
&= \boldsymbol{y}^T\boldsymbol{y} - \boldsymbol{y}^T X\boldsymbol{w} - (X\boldsymbol{w})^T\boldsymbol{y} + (X\boldsymbol{w})^T X\boldsymbol{w} && (3)\\
&= \boldsymbol{y}^T\boldsymbol{y} - \boldsymbol{y}^T X\boldsymbol{w} - \boldsymbol{w}^T X^T\boldsymbol{y} + \boldsymbol{w}^T X^T X\boldsymbol{w} && (4)\\
&= \boldsymbol{y}^T\boldsymbol{y} - (\boldsymbol{y}^T X\boldsymbol{w})^T - \boldsymbol{w}^T X^T\boldsymbol{y} + \boldsymbol{w}^T X^T X\boldsymbol{w} && (5)\\
&= \boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{w}^T X^T\boldsymbol{y} + \boldsymbol{w}^T X^T X\boldsymbol{w} && (6)
\end{aligned}$$

In Equations 3 and 6, we apply the identity $(AB)^T = B^T A^T$, while Equation 5 follows from the fact that $\boldsymbol{y}^T X\boldsymbol{w}$ is a scalar, so that its value equals its transpose.

Now let's take the derivative:

$$\begin{aligned}
&\nabla_{\boldsymbol{w}}\text{loss}(\boldsymbol{w}) && (7)\\
&= \nabla_{\boldsymbol{w}}\boldsymbol{y}^T\boldsymbol{y} - 2\nabla_{\boldsymbol{w}}\boldsymbol{w}^T X^T\boldsymbol{y} + \nabla_{\boldsymbol{w}}\boldsymbol{w}^T X^T X\boldsymbol{w} && (8)\\
&= -2X^T y + 2X^T Xw && (9)
\end{aligned}$$

Equation 9 relies on the following fact: $\nabla_{\boldsymbol{x}}\boldsymbol{x}^T A\boldsymbol{x} = 2A\boldsymbol{x}$.

Setting this derivative equal to zero yields a **closed-form** (i.e., analytical) solution to linear regression:

$$\begin{aligned}
-2X^T y + 2X^T Xw &= 0 && (10)\\
X^T Xw &= X^T y && (11)\\
w &= \underbrace{(X^T X)^{-1}X^T}_{\text{pseudo-inverse}} y && (12)
\end{aligned}$$

The matrix $X^T X$ might not be invertible. This matrix is only invertible when the columns of X are linearly independent, so that they span $\mathbb{R}^d$. The pseudo-inverse $(X^T X)^{-1}X^T$, however, always exists, and can be calculated via a **singular value decomposition**.

We have thus derived a $\boldsymbol{w}$ that satisfies the **first-order optimality condition**, namely $\nabla_{\boldsymbol{w}}\text{loss}(\boldsymbol{w}) = 0$. To conclude that $\boldsymbol{w}$ is a minimum, we also need to establish that it satisfies the **second-order optimality condition**, namely that $\nabla^2_{\boldsymbol{w}}\text{loss}(\boldsymbol{w}) \geq 0$. Indeed, $\nabla^2_{\boldsymbol{w}}\text{loss}(\boldsymbol{w}) = 2X^T X \geq 0$.

Alternative Derivation We describe an alternative shorter, geometric, and arguably more intuitive way, to derive the least-squares estimator. The only way the residual $\boldsymbol{y} - X\boldsymbol{w}$ can be zero is if $\boldsymbol{y}$ is a **linear combination** of the columns of X: i.e., if $\boldsymbol{y}$ lies in the **column span** of X. As most of the time it does not, our stated goal of minimizing the residual (equivalently, the total value of the squared residuals) can instead be understood as minimizing the distance from the residual to the column span of X. To minimize this distance, the residual should be **projected** onto the column span of X; that is, it should be **orthogonal** to each and every column of X: for all $j \in \{1, \ldots, n\}$, i.e., $(\boldsymbol{y} - X\boldsymbol{w}) \cdot X_j = \boldsymbol{0}_d$. Equivalently, but expressed more compactly in matrix notation, $(\boldsymbol{y} - X\boldsymbol{w})^T X = \boldsymbol{0}_d$. This requirement implies:

$$\begin{aligned}
X^T(\boldsymbol{y} - X\boldsymbol{w}) &= \boldsymbol{0}_d && (13)\\
X^T X\boldsymbol{w} &= X^T\boldsymbol{y} && (14)\\
\boldsymbol{w} &= \underbrace{(X^T X)^{-1}X^T}_{\text{pseudo-inverse}}\boldsymbol{y} && (15)
\end{aligned}$$

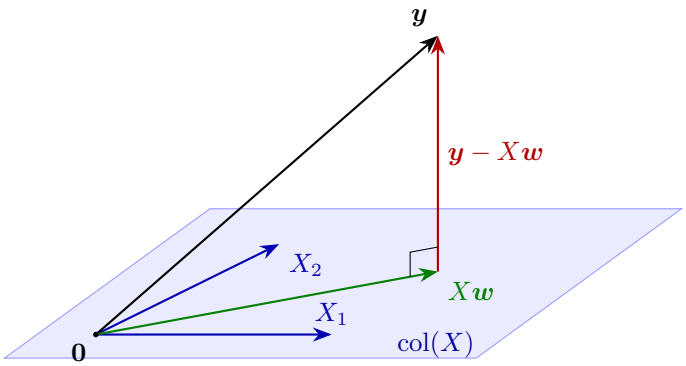


Figure 2: The vector $\boldsymbol{y}$ generally lies outside the column span of X. The closest point $X\boldsymbol{w}$ in $\mathrm{col}(X)$ is the orthogonal projection of $\boldsymbol{y}$, which makes the residual $\boldsymbol{y} - X\boldsymbol{w}$ orthogonal to every column X_j of X.

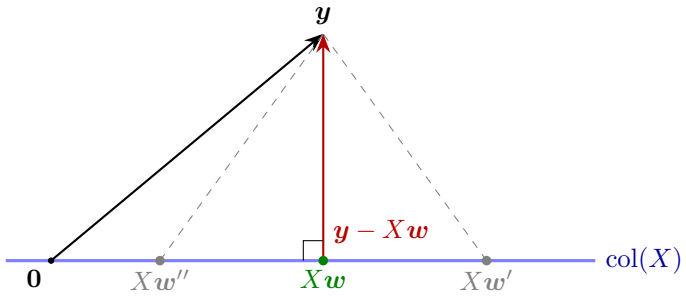


Figure 3: Among all points $X\boldsymbol{w}$ in $\mathrm{col}(X)$, the one whose residual is orthogonal to $\mathrm{col}(X)$ is closest to $\boldsymbol{y}$: any other choice ($X\boldsymbol{w}'$, $X\boldsymbol{w}''$) yields a longer residual, by the Pythagorean theorem.

2 Polynomial Regression

When $d = 1$, a **linear model** is a model in which each y_i is approximated by $mx_i + b$. Here, as in high school, m is the slope, while b is the y intercept, which shifts the line, and serves as the **bias** term in regression.

It is straightforward to incorporate a y intercept, and its accompanying bias, into our matrix notation. The trick is to append an extra dimension to the parameter vector $\boldsymbol{w} \in \mathbb{R}^d$, and to likewise append an extra feature/column to X, whose value is always 1, so that w_{d+1} represents the intercept:

$$X\boldsymbol{w} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} & 1 \\ x_{21} & x_{22} & \dots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{nd} & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{d+1} \end{bmatrix} = \begin{bmatrix} w_1 x_{11} + w_2 x_{12} + \ldots + w_d x_{1d} + w_{d+1} \\ w_1 x_{21} + w_2 x_{22} + \ldots + w_d x_{2d} + w_{d+1} \\ \vdots \\ w_1 x_{n1} + w_2 x_{n2} + \ldots + w_d x_{nd} + w_{d+1} \end{bmatrix}$$

In a simple regression of y on $X = \begin{bmatrix} \boldsymbol{x} \end{bmatrix}$, i.e., only one feature, the aforementioned trick is akin to appending $\boldsymbol{x}^0 = \mathbf{1}$ to X, resulting in $\begin{bmatrix} \boldsymbol{x} & 1 \end{bmatrix}$, or $\begin{bmatrix} 1 & \boldsymbol{x} \end{bmatrix}$. More generally, it is equally possible to append $\boldsymbol{x}$ raised to any other power p to X as well: e.g., $\begin{bmatrix} 1 & \boldsymbol{x} & \boldsymbol{x}^2 & \boldsymbol{x}^3 & \dots & \boldsymbol{x}^p \end{bmatrix}$. Therefore, **polynomial regression** generalizes linear regression! In theory, at least. In practice, the set of possible combinations of powers of features is

massive. Assuming just two features, say $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ and $p = 3$ yields the following (long) list of possibilities:[6]

$$1 \;\; \boldsymbol{x}_1 \;\; \boldsymbol{x}_1^2 \;\; \boldsymbol{x}_1^3 \;\; \boldsymbol{x}_2 \;\; \boldsymbol{x}_2^2 \;\; \boldsymbol{x}_2^3 \;\; \boldsymbol{x}_1\boldsymbol{x}_2 \;\; \boldsymbol{x}_1^2\boldsymbol{x}_2 \;\; \boldsymbol{x}_1\boldsymbol{x}_2^2 \;\; \boldsymbol{x}_1^2\boldsymbol{x}_2^2 \;\; \boldsymbol{x}_1^3\boldsymbol{x}_2 \;\; \boldsymbol{x}_1\boldsymbol{x}_2^3 \;\; \boldsymbol{x}_1^3\boldsymbol{x}_2^2 \;\; \boldsymbol{x}_1^2\boldsymbol{x}_2^3 \;\; \boldsymbol{x}_1^3\boldsymbol{x}_2^3$$

Feature selection is an important and difficult problem in machine learning. The deep learning revolution can in large part be attributed to its success at automatically identifying pertinent features.

3 Regularized Regression

Linear regression is a machine learning model with a strong bias, namely the decision to fit a *line* (as opposed to a curve) to data. Even with this inherent bias, linear regression models can have high variance. Another way to limit variance is to limit the range of the model parameters $\boldsymbol{w} \in \mathbb{R}^d$.

The p-**norm** of a vector $\boldsymbol{w} \in \mathbb{R}^d$, denoted $\|\boldsymbol{w}\|_p$, for some $p \geq 1$, is a way to gauge $\boldsymbol{w}$'s size.[7]

The most popular norm is the 2-norm, also called the **Euclidean norm**:

$$\|\boldsymbol{w}\|_2 = \sqrt{\sum_{i=1}^{d} w_i^2}$$

Other common choices include the 1-norm:

$$\|\boldsymbol{w}\|_1 = \sum_{i=1}^{d} |w_i|$$

and the ∞-norm:

$$\|\boldsymbol{w}\|_\infty = \max_{i \in \{1,\ldots,d\}} |w_i|$$

Figure 4 depicts the feasible sets when the p-norm of a vector in $\mathbb{R}^d$ is constrained to be less than or equal to 1, for $p \in \{1, 2, 1000, \infty\}$.

Recall the ordinary least squares (OLS) objective:

$$\text{loss}(\boldsymbol{w}) = (\boldsymbol{y} - X\boldsymbol{w})^T(\boldsymbol{y} - X\boldsymbol{w}) = \sum_{i=1}^{n} (\boldsymbol{y} - X\boldsymbol{w})_i^2$$

In other words,

$$\text{loss}(\boldsymbol{w}) = \|\boldsymbol{y} - X\boldsymbol{w}\|_2^2$$

Observe that OLS is an unconstrained optimization problem, as $\boldsymbol{w}$ can take on any value in $\mathbb{R}^d$. **Regularized regression** is a constrained optimization problem, with the same objective, but a limited domain for $\boldsymbol{w}$.

[6] I probably missed some!

[7] A norm is a function from $\mathbb{R}^d \to \mathbb{R}$ that satisfies the following three properties:

1. $\|\boldsymbol{x}\| > 0$, for all $\boldsymbol{x} \neq 0$
2. $\|\alpha\boldsymbol{x}\| = |\alpha|\|\boldsymbol{x}\|$, for all $\alpha \in \mathbb{R}$ and $\boldsymbol{x} \in \mathbb{R}^d$
3. $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$, for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$

The final property is called the **triangle inequality**, because the sum of the lengths of two sides of a triangle is at least that of the third.

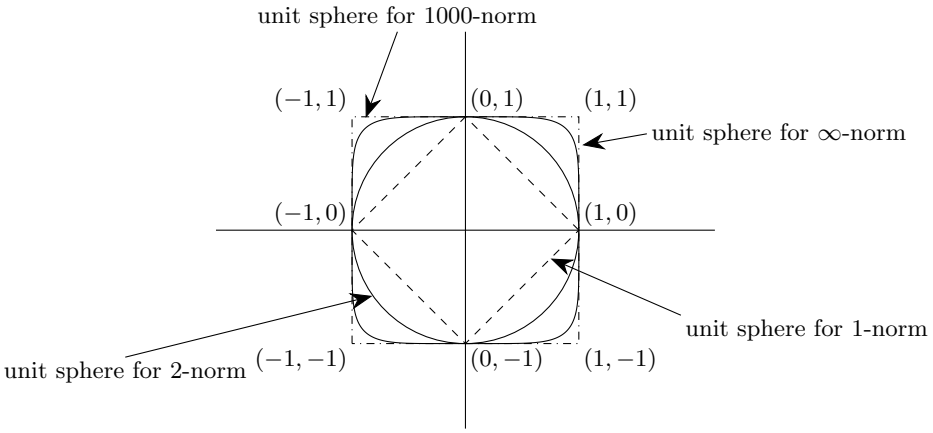


Figure 4: The feasible sets when the p-norm of a vector in $\mathbb{R}^d$ is constrained to be less than or equal to 1, for $p \in \{1, 2, 1000, \infty\}$. Original image source.

Ridge regression uses the 2-norm as a regularizer: for some $\beta > 0$,

$$\min_{\boldsymbol{w} \in \mathbb{R}^d} \|\boldsymbol{y} - X\boldsymbol{w}\|_2^2$$
$$\text{subject to } \|\boldsymbol{w}\|_2 \leq \beta$$

LASSO uses the 1-norm: for some $\beta > 0$,

$$\min_{\boldsymbol{w} \in \mathbb{R}^d} \|\boldsymbol{y} - X\boldsymbol{w}\|_2^2$$
$$\text{subject to } \|\boldsymbol{w}\|_1 \leq \beta$$

The choice of β is a choice of how much bias to include in the model.

Like OLS, ridge regression has a closed-form solution. This solution can be found by first reformulating the ridge regression constrained optimization problem in terms of its Lagrangian,[8] and then following the same steps as last time to solve OLS. The closed-form solution of ridge regression generalizes that of OLS:

$$\boldsymbol{w} = (X^T X + \lambda I)^{-1} X^T \boldsymbol{y}$$

On the other hand, LASSO does not have a closed-form solution. The difficulty arises from the fact that the absolute value function is not differentiable at zero. As a result, LASSO can be solved using a generalization of gradient descent called *subgradient* descent, as the subgradient exists everywhere. The subgradient, however, is not unique, so this algorithm is usually slower to converge than gradient descent.[9]

While both ridge regression and LASSO impose bias, and thereby limit variance, their behavior is notably different. Take, for example, the vectors $(1, 0)$ and $(1/\sqrt{2}, 1/\sqrt{2})$. While $\|(1, 0)\|_2 = \|(1/\sqrt{2}, 1/\sqrt{2})\|_2 = 1$, only $(1, 0)$ has 1-norm 1; $\|(1/\sqrt{2}, 1/\sqrt{2})\|_1 = 2/\sqrt{2} = \sqrt{2}$. As a result, **LASSO has a tendency to select features**, by completely zeroing out less important features, not just assigning them small coefficients.

Figure 5 is a depiction of ridge regression and LASSO. The OLS minimum is the dark black dot. As this point lies outside the feasible set, the optimal point for each problem lies on the boundary of the feasible set. But only for LASSO does it fall on a corner; the coefficient in the y direction is completely zeroed out.

[8] The λ in this expression is called a Lagrange multiplier, and can be interpreted as a penalty for violating the constraint.

[9] In practice, LASSO problems are in fact solved using an algorithm called **coordinate descent**, which proceeds, for example, in a round-robin fashion, optimizing one coordinate at a time.

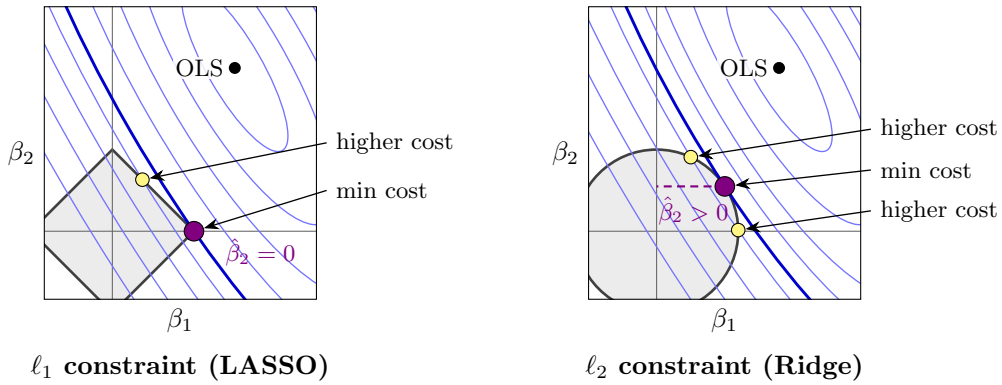


Figure 5: A depiction of the contour map for an OLS problem, as well as the feasible regions the LASSO and Ridge regression variants. There are an infinite number of contour lines; the figure depicts a few of them, including the ones that meet the respective feasible regions. LASSO prefers a zero coefficient for $\hat{\beta}_2$, while Ridge favors a non-zero setting. Original image source.